

Counting Bits Approach

338. Counting Bits — Approach, Code, and Detailed Explanation

We need to return an array where:

```
ans[i] = number of 1s in binary representation of i
```

For example:

```
0 → 0      → 0 ones
1 → 1      → 1 one
2 → 10     → 1 one
3 → 11     → 2 ones
4 → 100    → 1 one
5 → 101    → 2 ones
```

So for $n = 5$:

```
C++
[0, 1, 1, 2, 1, 2]
```

1. Brute Force Approach — $O(n \log n)$

For every number, repeatedly check its last bit:

```
C++
int countBits(int x) {
    int count = 0;
    while (x > 0) {
        count += x % 2;
        x /= 2;
    }
    return count;
}
```

Then:

```
C++
for (int i = 0; i <= n; i++) {
    ans[i] = countBits(i);
}
```

Complexity

Each number can take up to $\log n$ operations.

```
 $O(n \log n)$ 
```

But the question asks us to do better:

```
 $O(n)$ 
```

2. Optimal DP Approach — $O(n)$

The key observation is:

```
i = binary number
i / 2 = remove the last binary digit
```

For example:

```
5 = 101
5/2 = 2 = 10
```

And:

```
5 % 2 = 1
```

So:

```
number of 1s in 5
=
number of 1s in 2
```

```
+
last bit of 5
```

Therefore:

Formula

```
C++
ans[i] = ans[i / 2] + (i % 2);
```

Or using bit operations:

```
C++
ans[i] = ans[i >> 1] + (i & 1);
```

3. Understanding the Formula

Let's take:

```
i = 13
```

Binary:

```
13 = 1101
```

Now:

```
13 >> 1 = 110 = 6
```

So:

```
1101
↓ remove last bit
110
```

The last bit is:

```
13 & 1 = 1
```

Therefore:

```
ans[13] = ans[6] + 1
```

Since:

```
6 = 110
```

It has:

```
2 ones
```

So:

```
ans[13] = 2 + 1 = 3
```

Correct:

```
13 = 1101 → 3 ones
```

4. C++ Code

```
C++
class Solution {
public:
    vector<int> countBits(int n) {
        vector<int> ans(n + 1);

        ans[0] = 0;

        for (int i = 1; i <= n; i++) {
            ans[i] = ans[i >> 1] + (i & 1);
        }

        return ans;
    }
};
```

```
}  
};
```

5. Dry Run for $n = 5$

Initially:

```
ans = [0, 0, 0, 0, 0, 0]
```

i = 1

```
1 >> 1 = 0  
1 & 1 = 1
```

So:

```
ans[1] = ans[0] + 1  
        = 0 + 1  
        = 1
```

Array:

```
[0, 1, 0, 0, 0, 0]
```

i = 2

Binary:

```
2 = 10
```

```
2 >> 1 = 1  
2 & 1 = 0
```

So:

```
ans[2] = ans[1] + 0  
        = 1 + 0  
        = 1
```

Array:

```
[0, 1, 1, 0, 0, 0]
```

i = 3

Binary:

```
3 = 11
```

```
3 >> 1 = 1  
3 & 1 = 1
```

So:

```
ans[3] = ans[2] + 1  
        = 1 + 1  
        = 2
```

Array:

```
[0, 1, 1, 2, 0, 0]
```

i = 4

Binary:

```
4 = 100
```

```
4 >> 1 = 2  
4 & 1 = 0
```

So:

```
ans[4] = ans[3] + 0  
        = 2 + 0  
        = 2
```

Array:

```
[0, 1, 1, 2, 1, 0]
```

i = 5

Binary:

```
5 = 101
```

```
5 >> 1 = 2
5 & 1 = 1
```

So:

```
ans[5] = ans[2] + 1
        = 1 + 1
        = 2
```

Final:

```
[0, 1, 1, 2, 1, 2]
```

Visual Pattern

i	Binary	i >> 1	ans[i >> 1]	i & 1	ans[i]
0	0	-	-	-	0
1	1	0	0	1	1
2	10	1	1	0	1
3	11	1	1	1	2
4	100	2	1	0	1
5	101	2	1	1	2
6	110	3	2	0	2
7	111	3	2	1	3

Why does this work?

Every binary number can be represented as:

```
[all bits except last bit] + [last bit]
```

For example:

```
13 = 1101
```

Remove the last bit:

```
110
```

which is:

```
13 >> 1 = 6
```

Then simply add whether the removed last bit was 0 or 1:

```
C++
ans[i] = ans[i >> 1] + (i & 1);
```

Meaning

```
C++
i >> 1
```

→ removes the last binary bit.

```
C++
i & 1
```

→ tells us whether the last bit is 0 or 1.

Another Very Popular Approach

There is another beautiful formula:

```
C++
ans[i] = ans[i & (i - 1)] + 1;
```

Why?

The expression:

```
C++
i & (i - 1)
```

removes the **rightmost set bit (1)**.

Example:

```
i = 12
12    = 1100
11    = 1011
----- &
      1000
```

So:

```
1100 → 1000
```

One 1 was removed.

Therefore:

```
number of set bits in i
=
number of set bits in (i & (i - 1))
+
1
```

Code:

```
C++
class Solution {
public:
    vector<int> countBits(int n) {
        vector<int> ans(n + 1, 0);

        for (int i = 1; i <= n; i++) {
            ans[i] = ans[i & (i - 1)] + 1;
        }

        return ans;
    }
};
```

Which approach should you remember?

I recommend remembering this one:

```
C++
ans[i] = ans[i >> 1] + (i & 1);
```

Because it directly means:

```
Current number
  ↓
Remove last bit
  ↓
Use already calculated answer
  +
Add the last bit
```

Complexity

```
Time:  O(n)
Space: O(n)
```

And it works in **one pass**, exactly as required by the follow-up.

Yes — let's forget the DP formula for a moment. The easiest way to understand it is through **binary numbers**.

We want to find:

How many 1s are present in every number from 0 to n?

Let's take numbers from 0 to 7.

Number	Binary	Number of 1s
0	0	0
1	1	1
2	10	1
3	11	2
4	100	1
5	101	2
6	110	2
7	111	3

Now let's understand the main trick:

```
C++
ans[i] = ans[i >> 1] + (i & 1);
```

Think about 5

Binary of 5 is:

```
5 = 101
```

We want to count the 1s.

Instead of counting everything again, **remove the last bit**.

```
101
↓ remove last bit
10
```

And:

```
101 = 5
10  = 2
```

So after removing the last bit from 5, we get 2.

In C++:

```
C++
5 >> 1
```

means:

```
101 → 10
5   → 2
```

Now ask:

How many 1s are already in 10?

We already calculated it!

```
2 = 10
Number of 1s = 1
```

Now we only need to check the bit we removed.

```
101
last bit = 1
```

So:

```
ones in 101
=
ones in 10
+
last bit
2
```

```
=
1
+
1
```

Therefore:

```
C++
ans[5] = ans[2] + 1;
```

The entire logic in one picture

Take any binary number:

```
1011
-----
first part + last bit
```

We can break it into:

```
101 | 1
```

The number of 1s in 1011 is:

```
(number of 1s in 101)
+
(last bit)
```

For example:

```
1011
101 has → 2 ones
last bit → 1
Total = 2 + 1 = 3
```

That is exactly what this does:

```
C++
ans[i] = ans[i >> 1] + (i & 1);
```

Now understand both parts separately

Part 1: `i >> 1`

This removes the **last binary digit**.

Examples:

```
1 = 1      → 0
2 = 10     → 1
3 = 11     → 1
4 = 100    → 10
5 = 101    → 10
6 = 110    → 11
7 = 111    → 11
```

So:

```
5 = 101
5 >> 1
101 → 10
5 → 2
```

Basically:

```
C++
i >> 1
```

is the same as:

```
C++
i / 2
```

for non-negative integers.

Part 2: i & 1

This checks the **last binary bit**.

Example: Even number

```
6 = 110
last bit = 0
```

```
C++
6 & 1 = 0
```

Example: Odd number

```
5 = 101
last bit = 1
```

```
C++
5 & 1 = 1
```

So:

```
i & 1
Even number → 0
Odd number  → 1
```

Now combine them

Let's calculate `ans[6]`.

```
6 = 110
```

Step 1: Remove the last bit

```
110 → 11
6 >> 1 = 3
```

We already know:

```
3 = 11
Number of 1s = 2
```

So:

```
ans[3] = 2
```

Step 2: Check the removed bit

```
110
last bit = 0
```

Therefore:

```
ans[6] = ans[3] + 0
        = 2 + 0
        = 2
```

Correct:

```
6 = 110 → two 1s
```

Full dry run from 0 to 7

Start:

```
ans[0] = 0
```

i = 1

1 = 1

Remove last bit:

1 → 0

Last bit:

1

So:

```
ans[1] = ans[0] + 1
        = 0 + 1
        = 1
```

i = 2

2 = 10

Split it:

1 | 0

The first part 1 has:

1 one

Last bit is:

0

So:

```
ans[2] = ans[1] + 0
        = 1
```

i = 3

3 = 11

Split:

1 | 1

First part:

1 → 1 one

Last bit:

1

Therefore:

```
ans[3] = ans[1] + 1
        = 1 + 1
        = 2
```

i = 4

4 = 100

Split:

10 | 0

10 is number 2.

We already know:

ans[2] = 1

Last bit:

```
0
```

Therefore:

```
ans[4] = ans[2] + 0
        = 1
```

$i = 5$

```
5 = 101
```

Split:

```
10 | 1
```

10 is number 2.

```
ans[2] = 1
```

Last bit:

```
1
```

Therefore:

```
ans[5] = ans[2] + 1
        = 1 + 1
        = 2
```

The most important idea

Look at this pattern:

```
5 = 101
    ↑
    remove this last bit
After removing it:
101 → 10
5   → 2
```

We have **already calculated the answer for 2** because we are going from:

```
0 → 1 → 2 → 3 → 4 → 5 ...
```

So we don't need to count all bits of 5 again.

We simply say:

"I already know how many 1s are in 2. Let me just check whether the new last bit of 5 is 0 or 1."

Therefore:

```
answer of current number
=
answer of previous smaller binary part
+
new last bit
```

And that becomes:

```
C++
ans[i] = ans[i >> 1] + (i & 1);
```

Final code

```
C++
class Solution {
public:
    vector<int> countBits(int n) {
        vector<int> ans(n + 1);

        for (int i = 1; i <= n; i++) {
            ans[i] = ans[i >> 1] + (i & 1);
        }
    }
};
```

```
    } return ans;  
};
```

Just remember this meaning:

```
C++  
i >> 1
```

→ Remove the last binary bit

```
C++  
i & 1
```

→ Get the last binary bit

So:

```
Current answer  
=  
Answer after removing last bit  
+  
The removed last bit
```

That is the complete DP logic.

ChatGPT can make mistakes. Check important info.